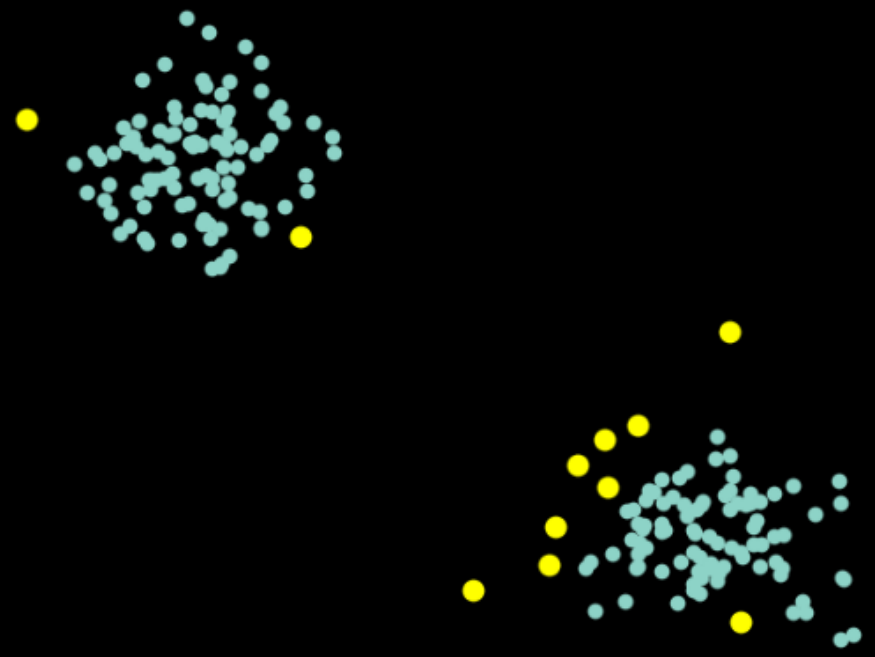
Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

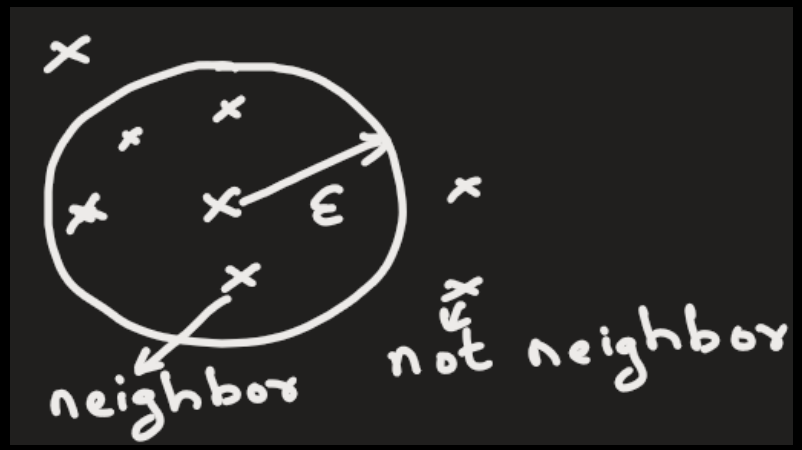
In DBSCAN, it clusters data points based on their density and identifies outliers as points not belonging to any cluster.

i.e. DBSCAN labels points without enough nearby neighbors as outliers.

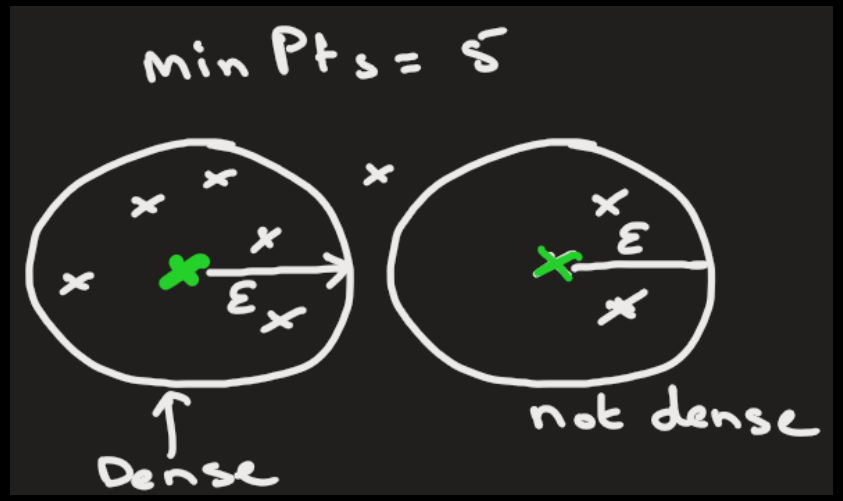


DBSCAN uses **two parameters**:

1) ε (Epsilon) — Neighborhood Radius
Maximum distance to consider neighbors.
Think of it as a **circle radius** around a point.
"Which points is close enough to me?"



2) MinPts — Minimum Points
Minimum number of neighbors required to form a dense region.



Choosing MinPts (Rough Guideline):

- **MinPts ≥ dimensions + 1**

Example:
3D data → MinPts ≈ 4 - 5

Transaction_time	Transaction_amount	Card_type
2 pm	2000	Visa
1 pm	9000	Mastercard
12:30 am	890	Visa
…	…	…

- **MinPts = 2 x dimensions**

Example:
3D data → MinPts = 2 x 3 = 6

So, high-dimensional means larger minPts

Choosing Epsilon :
Use Elbow method.

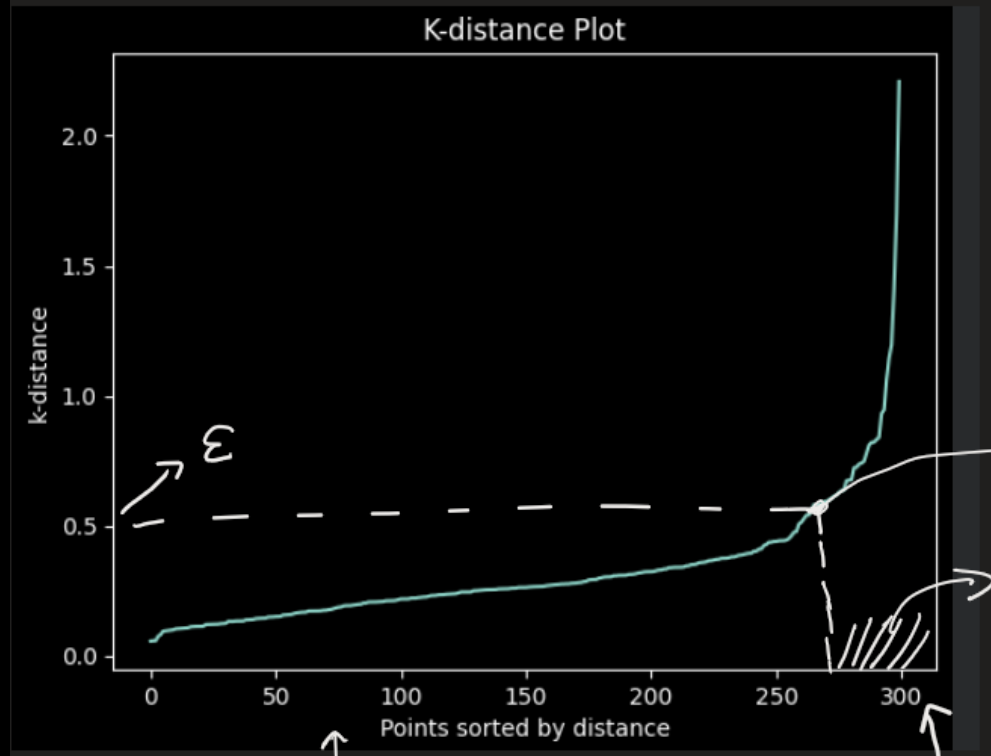
1) Suppose we choose K=3

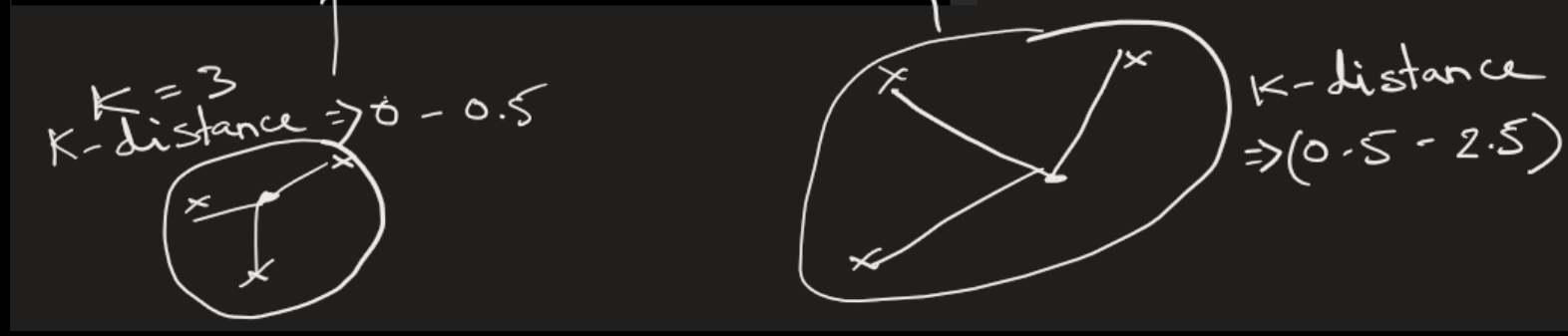
2) For all points, find the distance to K nearest neighbor
i.e. How far must point go to find **k neighbors ?**

3) Sort the distance

4) Plot the points with K-distance

5) Find elbow point and read epsilon
Points that are towards right of Elbow are good candidates for outliers.



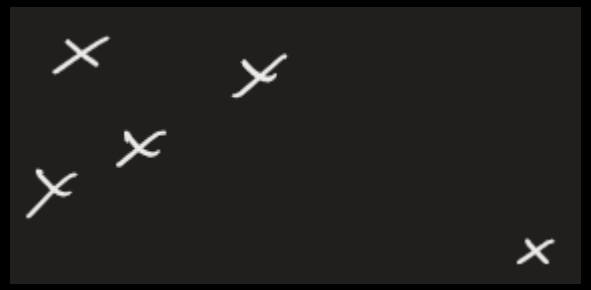


Note:
DBSCAN is distance-based → **scale data first**.

Application:

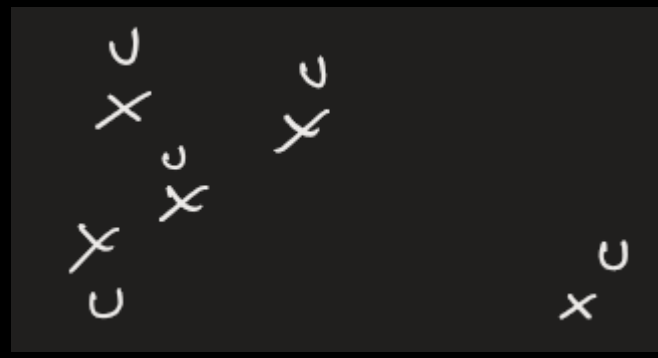
- Fraud detection
- Network intrusion detection
- Sensor anomaly detection
- Customer behavior analysis
- Spatial/geographical data

DBSCAN Algorithm



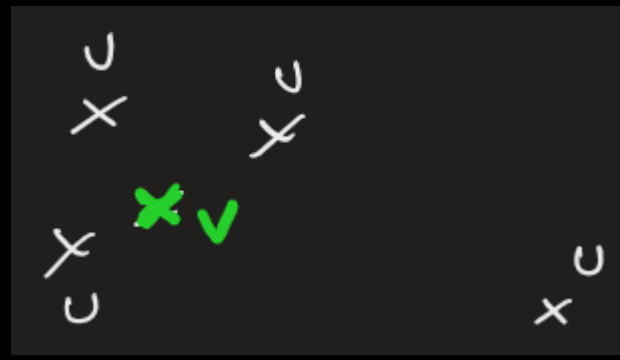
Step 1 — Mark all points as UNVISITED

```
for each point p:
    label[p] = UNVISITED
```



Step 2 — Pick an Unvisited Point

Select a random point **p**.
Mark it as visited.

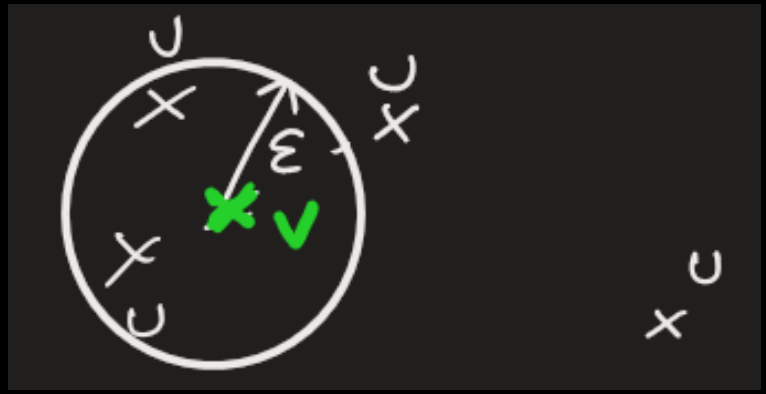


DBSCAN Algorithm

Step 3 — Find Neighbors

Compute:

neighbors = points within distance ε of p

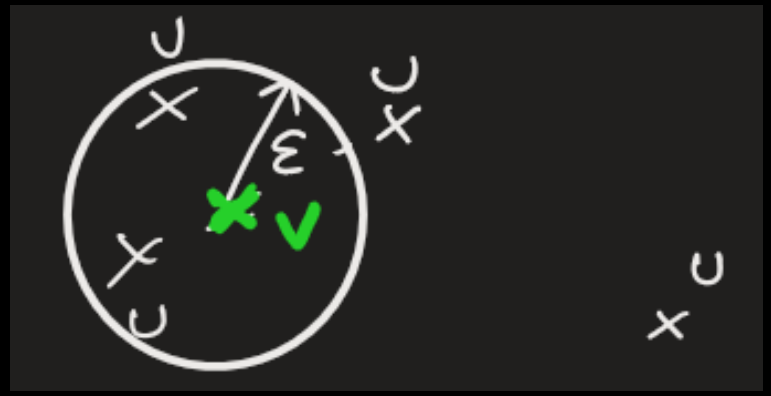


DBSCAN Algorithm

Step 4 — Check Density Condition

Case A: Not enough neighbors

```
if len(neighbors) < MinPts:
    mark p as NOISE (outlier candidate)
```



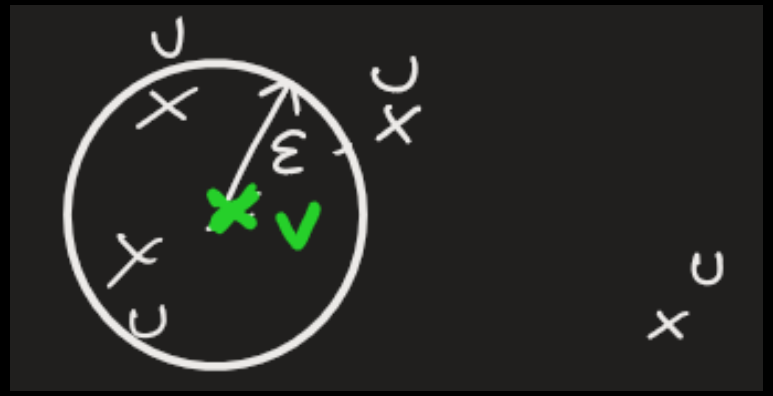
These are temporary label (may change later).

Case B: Dense region found

```
else:
    create new cluster
    p becomes CORE point
```

DBSCAN Algorithm

Step 5 — Expand Cluster (Density Reachability)



For each neighbor q:

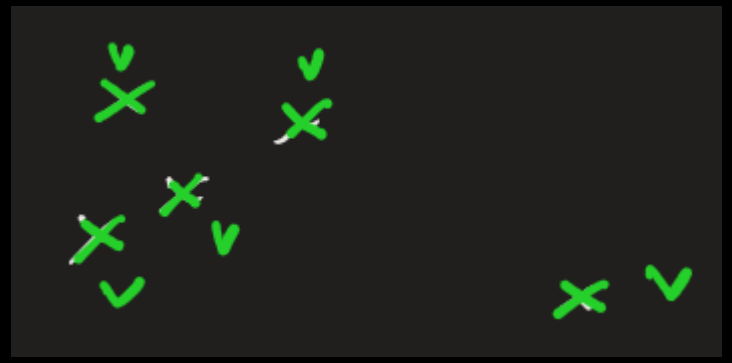
1) If q is unvisited → visit it.
2) Find neighbors of q.
3) If q has ≥ MinPts neighbors:
   merge neighbors (cluster grows).
4) Assign q to the cluster.

This expansion continues recursively.

DBSCAN Algorithm

Step 6 — Repeat

Continue until all points are visited.



Step 7 — Final Outlier Detection

After clustering:

points not assigned to any cluster

↓

label = NOISE (-1)

↓

OUTLIERS

DBSCAN Algorithm

A point is an **outlier** if:

It is NOT density-reachable from any core point.

EXTRA

3. Clustering-Based Methods:

- **Density-Based Spatial Clustering of Applications with Noise (DBSCAN):** In DBSCAN, clusters data points based on their density and identifies outliers as points not belonging to any cluster.

- **Hierarchical clustering:** Hierarchical clustering involves building a hierarchy of clusters by iteratively merging or splitting clusters based on their similarity. **Outliers can be identified as clusters containing only a single data point or clusters significantly smaller than others.**

4. Other Methods:

- **Isolation Forest:** Isolation forest randomly isolates data points by splitting features and identifies outliers as those isolated quickly and easily.

- **One-class Support Vector Machines (OCSVM):** One-Class SVM learns a boundary around the normal data and identifies outliers as points falling outside the boundary.

Outlier example: TODO